

Minor Project Defense — Q&A Preparation

Fault Detection and Bit-Error Rate Diagnostics in Optical Wireless Links Using Edge-AI

This document collects likely defense questions organized by domain, with simple, spoken-style answers you can internalize and explain in your own words.

1. General / Project-Level Questions

Q1. What problem does your project solve, in one sentence? Optical wireless links break down for different physical reasons (blockage, misalignment, filtering, ambient light), and a normal receiver only tells you "the link is bad" via BER — not *why*. Your system figures out the *why*, locally, on the receiver itself, without sending data to a computer or the cloud.

Q2. Why does this matter? Why not just look at BER alone? BER only tells you *how bad* the link is, not *what caused it*. Two very different problems — an obstruction versus bright ambient light — can both increase BER. If you know the cause, you (or an automated system in the future) can react appropriately: adjust alignment, add a filter, or reposition the receiver. BER alone can't distinguish these causes; that's the gap your work fills.

Q3. What is "Edge-AI" and why did you use it here instead of sending data to a laptop/cloud? Edge-AI means running the machine-learning model directly on the microcontroller instead of on an external server. Benefits: no network dependency, lower latency (decision happens instantly after each communication cycle), and the receiver stays autonomous — useful for a communication node that may be deployed somewhere without internet access.

Q4. What are the four classes your system detects?

Class 0: Normal condition

Class 1: Optical filter attenuation (signal weakened by a filter in the path)

Class 2: High ambient light (background light interferes with detection)

Class 3: Severe obstruction / misalignment (path blocked or badly misaligned)

Q5. What are the four features your model uses, and why these four? Signal Voltage, BER, SNR, and Ambient Light. Each captures a different aspect of the link: voltage = strength of the received signal, BER = actual communication reliability, SNR = how clean the signal is relative to background noise, ambient light = the surrounding optical environment. No single one of these can separate all four fault types alone — together they give the classifier enough information to distinguish causes, not just severity.

Q6. What is the overall system architecture? Two ESP32 boards: a transmitter node and a receiver node, communicating over free-space optical light (a laser). The transmitter sends a known payload using On-Off Keying. The receiver recovers the payload, computes the four diagnostic features from the same communication cycle, and feeds them into a small quantized neural network running on-device (TensorFlow Lite Micro) to output one of the four classes.

Q7. What is your project's main contribution / novelty compared to existing work? Prior work (cited in your literature review) mostly used ML to *optimize* optical links (adaptive transmission, channel

prediction) — not to *diagnose the physical cause* of degradation. You combined four complementary features and deployed a lightweight classifier directly on embedded hardware to identify the *type* of fault in real time, without external compute.

Q8. What are the current limitations of your system?

Only 4 predefined fault classes — anything else isn't recognized.

It only diagnoses; it doesn't correct the fault (no auto-realignment).

Data rate is limited by ADC sampling speed and the 5-sample majority-vote overhead.

If the link is *completely* blocked, synchronization itself fails, so no feature vector can even be formed — the system can't classify a total blackout, only degraded-but-still-synchronized links.

Q9. What would you improve if you had more time? Larger/more diverse dataset, a 5th "complete link failure" class or separate signal-presence detector, better line coding for efficiency, and eventually closing the loop with corrective actions (e.g., automatic realignment or power control) instead of just diagnosis.

2. Hardware Domain

Q10. Why did you choose a laser + photodiode instead of an LED? A laser gives a narrow, directional beam which is easier to control and modulate cleanly for a point-to-point link, and it's simple to switch fully on/off (OOK) using a transistor. The HW-493 module was low-cost and easy to interface with a GPIO-driven switch.

Q11. Why is the laser driven through a transistor instead of directly from the ESP32 GPIO? GPIO pins can only source a small amount of current (a few tens of mA) at 3.3V logic level, which isn't enough or safe for reliably driving the laser module. The 2N2222 transistor acts as an electronic switch: a small GPIO current into its base (through the 1k Ω resistor) controls a much larger current through the collector-emitter path to switch the laser fully on or off. This also protects the microcontroller from back-current or voltage mismatches.

Q12. Why is there a 1k Ω resistor between GPIO 23 and the transistor base? It limits the base current so the GPIO isn't overloaded and the transistor is driven safely into saturation (fully "on") without excessive current damaging either the GPIO or transistor.

Q13. Why BPW34 photodiode for the receiver? It's a PIN photodiode with a spectral response well-suited to visible/near-infrared light including the ~650nm laser wavelength, fast enough response for kbps-level modulation, and cheap/easy to interface — appropriate for a low-cost prototype.

Q14. Why did you use an LM358 op-amp, and why in this feedback configuration (100k Ω resistor + 103 capacitors)? The photodiode produces a very small current when hit by light, not directly usable by the ADC. The LM358 is configured to convert and amplify this small photocurrent-driven signal into a measurable voltage swing.

The **100k Ω feedback resistor** sets the gain of the amplifier — it determines how much the small diode signal gets amplified into a usable voltage range for the ADC.

The **103 capacitor across the feedback path** (10nF) acts as a low-pass filter in the feedback loop — it smooths out high-frequency noise/spikes and prevents oscillation, giving a cleaner signal.

The **103 capacitor on the supply pin** is a decoupling capacitor — it filters noise on the LM358's power supply so voltage ripples don't corrupt the amplified signal. In short: resistor = gain, feedback cap = noise filtering/stability, supply cap = clean power.

Q15. Why LM358 specifically and not another op-amp? It's a widely available, low-cost, dual op-amp that works well from a single 3.3V–5V supply (important since your circuit runs off 3.3V), making it convenient and cheap for a receiver front-end without needing a dual/split power supply.

Q16. Why is the photodiode connected in reverse-bias (cathode to +3.3V, anode to the amplifier input)? Reverse-biasing a photodiode reduces its junction capacitance and improves its response speed, which is important for correctly detecting the fast on/off optical transitions at 1000 bits/sec. It also makes the photocurrent flow in a direction convenient for the transimpedance-style front end feeding the LM358.

Q17. Why ESP32 for transmitter and ESP32-S3 specifically for the receiver? The receiver needs to run ADC sampling, feature computation, AND a TensorFlow Lite Micro neural network inference — this needs more processing power/memory. The ESP32-S3 has better performance and is well supported by TensorFlow Lite Micro tooling, making it more suitable for on-device inference than a plain ESP32, which is sufficient for the simpler transmitter role.

Q18. What does GPIO 4 do on the receiver, and why that pin? It's configured as the ADC input pin that reads the conditioned (amplified) analog output of the LM358, so the microcontroller can digitize the optical signal for processing.

Q19. Why do the transmitter and receiver share a common ground? For any analog/digital signal referencing to work correctly, both circuits need the same 0V reference point. Without a common ground, the voltage levels seen by the ADC would be meaningless/unstable relative to the transmitter's switching.

3. Communication / Firmware Domain

Q20. What is OOK (On-Off Keying) and why did you choose it? OOK represents bit '1' as the laser being ON and bit '0' as OFF. It's the simplest possible intensity-modulation scheme for optical communication — very easy to implement with a single GPIO and transistor switch, which fits a low-cost embedded prototype.

Q21. What is NRZ-L line coding and why use it over something like Manchester encoding? NRZ-L keeps the light at a constant level for the entire bit duration (no transition within a bit). You initially considered Manchester encoding (which encodes each bit as a transition, e.g., high-to-low or low-to-high mid-bit) but dropped it because it requires precise half-bit timing synchronization, which added unnecessary complexity for this prototype. NRZ-L is simpler to generate and decode with basic threshold sampling.

Q22. Why 1000 bits/sec? Why not faster? The data rate is constrained by how fast the ADC can reliably sample and by the overhead of taking 5 samples per bit for majority voting. 1000 bps was a stable operating point that balanced reliable bit recovery against the added complexity of a faster link.

Q23. How does the receiver know when a transmission starts (synchronization)? The transmitter sends a distinct synchronization pulse (100ms) before the payload, followed by a 20ms guard interval. The

receiver continuously monitors the incoming signal for this specific pattern; once detected, it establishes a timing reference and starts sampling the payload bits after accounting for the guard interval.

Q24. Why is there a guard interval between the sync pulse and the payload? It creates a clear separation so the receiver doesn't accidentally interpret the tail end of the sync pulse as the first payload bit — it gives a safety margin for the receiver to finish processing the sync event before real data begins.

Q25. What is majority voting for bit recovery, and why 5 samples? Instead of trusting a single ADC reading per bit (which could be corrupted by noise or a transition edge), the receiver takes 5 samples spread across the middle 60% of each 1ms bit interval (200–800µs) and compares each against a threshold (1700 ADC counts). Whichever value (HIGH/LOW) appears in the majority of the 5 samples is taken as the final bit. This reduces the chance that one noisy/erroneous sample flips the whole bit decision.

Q26. Why sample only the middle 60% of the bit interval, not the whole thing? The edges of a bit period are more likely to be affected by transition effects from the previous/next bit (rise/fall time of the laser and photodiode response). Sampling the central region avoids these edge artifacts and gets a more stable reading.

Q27. How is the detection threshold (1700 ADC counts) chosen? It's set based on the observed ADC output range separating a "light on" state from a "light off"/ambient baseline state during testing — essentially calibrated from your amplifier's output range so it reliably distinguishes HIGH vs LOW.

Q28. How is BER actually calculated in firmware? Because the transmitted payload is a known, fixed text, the receiver has a reference copy of what it should have received. After decoding, it compares each recovered bit with the corresponding expected bit, counts mismatches, and divides by total bits sent: $BER = (\text{number of wrong bits}) / (\text{total bits})$. No external equipment is needed since the "ground truth" is already known.

Q29. How is ambient light measured, and why during a "transmitter OFF" interval? Right before/around the sync period, the transmitter is briefly not transmitting. During this window, whatever the photodiode detects is purely background/ambient light, not the intended signal. Averaging ADC samples in this window gives the ambient-light feature, and its variation is also used to estimate background noise (used in SNR).

Q30. How is Signal Voltage computed? The receiver takes the ADC reading during signal reception, subtracts the ambient baseline measured earlier (signal = raw reading – ambient), and converts the ADC counts to actual voltage using the ADC's known conversion scale.

Q31. How is SNR calculated? $SNR (dB) = 20 \cdot \log_{10}(S / \sigma_n)$, where S is the effective signal (signal voltage minus ambient) and σ_n is the standard deviation of the ambient/background measurements (used as the noise estimate). A higher SNR means the real signal stands out more clearly above background fluctuation.

Q32. Why did you remove the DHT22 sensor and switch from a 5000-bit deterministic sequence at 500bps to a known fixed text at 1000bps? These were changes made during development for practical reasons: the DHT22 (a temperature/humidity sensor) wasn't needed once the diagnostic features came directly from the optical signal itself, and the known fixed-text payload at 1000bps gave a simpler, more

reliable reference for BER calculation and a faster/cleaner communication cycle than the original longer bit-sequence design.

4. Mathematical / Theory Domain

Q33. What's the difference between BER and SNR, conceptually? BER is a *communication-layer* result — did the actual bits get through correctly? SNR is a *physical-layer* measure of signal quality — how strong is the real signal compared to background noise, independent of whether bits were decoded correctly. A link can have moderate SNR but still recover bits perfectly (BER=0), or a small SNR drop can push BER up sharply — that relationship is part of what your model learns.

Q34. Why express SNR in decibels? Decibels compress a wide range of ratios (from very large to very small signal-to-noise ratios) into a more manageable, human-readable scale using a logarithm, and it's the standard convention in communication engineering.

Q35. Explain the feature vector [Vsignal, BER, SNR_dB, L_ambient] — why combine physical and communication-layer numbers together? No single feature uniquely identifies the fault type. For instance, low signal voltage alone could mean attenuation OR obstruction — but combined with ambient light and SNR, the classifier can tell them apart, because each fault type produces a distinct *pattern* across all four features together, not just one.

Q36. What do Precision, Recall, and F1-score mean in your context?

Precision (per class): of all times the model *said* "Class X", how often was it actually Class X? (Are your positive predictions trustworthy?)

Recall: of all the actual Class X cases, how many did the model correctly catch? (Are you missing real cases?)

F1-score: the balance between precision and recall in one number — useful when you care about both false alarms and missed detections. Example from your results: Class 2 (High Ambient Light) had perfect precision/recall (1.00), meaning it was very cleanly separable; Class 3 (Severe Obstruction) had lower recall (0.26) before override rules were applied, meaning the raw model sometimes missed it, which the deterministic override helped fix.

Q37. What's a confusion matrix and what did yours show? It's a table comparing actual class vs. predicted class, showing where the model gets confused. Your model showed the most confusion between Class 1 (Optical Filter Attenuation) and Class 2 (High Ambient Light) — logical, because both conditions can reduce signal voltage and disturb ambient light readings in overlapping ways, making them harder to separate purely from these four features.

5. Machine Learning / Dataset Domain

Q38. How was your dataset built? You physically ran the hardware link under four controlled conditions (normal, optical filter attenuation, high ambient light, severe obstruction/misalignment), recorded the resulting four-feature telemetry for each run, and labeled each observation with its true condition. This makes it a real hardware-derived dataset, not simulated.

Q39. Was your dataset balanced across classes? No — class distribution was uneven (Normal ~80, Attenuation ~165, High Ambient Light ~50, Misalignment ~95 samples, per Figure 4). This imbalance is worth acknowledging: classes with fewer samples (like High Ambient Light) are harder to generalize on reliably, though in your case that particular class actually classified very well.

Q40. Why did you need data preprocessing before training? The four features have very different units and numeric ranges (voltage in volts, BER as a fraction, SNR in dB, ambient light in lux/counts). Without scaling them to a comparable range, features with larger raw numbers could dominate training. Preprocessing parameters (e.g., mean/scale) were computed only from training data and then applied consistently to validation/test/embedded data, to avoid "leaking" information from evaluation data into training.

Q41. What type of model did you train, and why a neural network over other classifiers? A compact neural network, compared against a Random Forest baseline. The neural network (78.48% accuracy) performed comparably to Random Forest (77.22%), but the key reason for choosing the neural network is that it can be converted and quantized into TensorFlow Lite Micro format for embedded deployment — Random Forests don't have an equivalent lightweight embedded deployment path on microcontrollers as readily.

Q42. How did you validate/evaluate the model properly (avoiding overfitting)? Data was split into training, validation, and test sets. Training accuracy/loss were tracked alongside validation accuracy/loss across epochs (Figure 6) to check the model wasn't just memorizing training data. Final performance was then measured on an independent test set using accuracy, precision, recall, F1, and confusion matrix.

Q43. What does the training/validation loss-accuracy curve show, and was there overfitting? Both training and validation loss decrease and accuracy increases together, converging around similar values by later epochs (Figure 6) — this indicates the model is generalizing reasonably rather than overfitting drastically, since validation performance tracks training performance instead of diverging.

Q44. What is quantization, and why did you apply it? Quantization converts the model's internal numbers from 32-bit floating point (FP32) to 8-bit integers (INT8). This drastically reduces model size and computational cost, which is essential because microcontrollers like the ESP32-S3 have very limited memory and no dedicated floating-point-heavy hardware compared to a PC/GPU.

Q45. What was the accuracy cost of quantization? Accuracy dropped from 78.48% (floating point) to 74.68% (quantized INT8) — about a 3.8 percentage-point reduction — in exchange for a very small model (3.32 KB), which is a reasonable trade-off for enabling on-device execution within microcontroller constraints.

Q46. Why is the model so small (3.32 KB)? Isn't that too simple to be a "real" neural network? The task only has 4 simple numeric inputs and 4 output classes, so it doesn't need a large/deep network — a small model can already capture the relationship. Bigger isn't better here: memory and inference speed matter more, and the compact model still achieved good accuracy while fitting comfortably in the ESP32-S3's constrained memory.

Q47. What is TensorFlow Lite Micro and how does it fit into your firmware? It's a version of TensorFlow Lite designed to run neural network inference on microcontrollers with minimal memory/no OS. After the model was converted and quantized, it was embedded directly into the receiver's C/C++ firmware; the

firmware initializes the interpreter, allocates memory ("tensor arena"), feeds in the preprocessed feature vector after each communication cycle, and reads out the predicted class — all locally on the ESP32-S3.

Q48. What is the "deterministic override rule" you mention in the results, and why did you need it? It's a rule layered on top of the neural network: for very clear-cut cases (e.g., extremely low BER and healthy signal voltage → force Class 0 "Normal"; or clearly negative SNR / high BER indicating signal loss → force Class 3 "Severe Obstruction"), the firmware directly assigns the obviously-correct class rather than relying purely on the ML model's raw softmax output. This improves reliability at the extremes, where the training data may have been sparse or where the model's raw confidence dipped, ensuring the system doesn't misclassify very obvious conditions.

Q49. Isn't using override rules "cheating" on the ML results? It's not cheating — it's a common practical technique in embedded/safety-relevant ML systems called a hybrid rule+ML approach. The neural network still does the core work of distinguishing the more ambiguous middle-ground cases (like attenuation vs. ambient light); the override just adds robustness for extreme, unambiguous cases where a wrong classification would be especially costly and where the physics makes the answer obvious anyway.

Q50. Why did the model most often confuse "Optical Filter Attenuation" and "High Ambient Light"? Both conditions can lower signal voltage and disturb the ambient light readings in similar directions — attenuation reduces the signal seen at the photodiode, while excess ambient light raises the baseline/reduces contrast, and both indirectly show up as changes in your 4 features in overlapping ways. This is a known limitation, and future work suggests adding more discriminating features (e.g., temporal/statistical signal variation) to separate them better.

Q51. Why does Class 3 (Severe Obstruction) show very high confidence (0.96) in the deployment heatmap while others are lower? Severe obstruction produces an extreme, unambiguous signature (very high BER, strongly negative SNR) that's clearly different from the other three conditions, plus the deterministic override reinforces it — so the system is very confident and accurate for this class specifically, unlike the more subtle boundary between attenuation and ambient light.

6. Results / Comparative Analysis Domain

Q52. Summarize your key quantitative results.

Under normal conditions: BER = 0, signal voltage $\approx 0.12\text{V}$, SNR $\approx 10.5\text{ dB}$.

Under severe obstruction: BER > 0.35 , SNR as low as -20 dB .

Neural network (floating point): 78.48% test accuracy.

Random Forest baseline: 77.22% test accuracy.

Quantized INT8 model: 74.68% test accuracy, 3.32 KB size.

Q53. Why compare against Random Forest at all? To justify the choice of neural network — you wanted to show the neural network isn't sacrificing much (or any) accuracy compared to a well-known, strong traditional ML baseline, while gaining the major practical benefit of straightforward quantization and embedded deployment that Random Forest doesn't offer as easily.

Q54. In the most severe obstruction case, the receiver couldn't classify at all — why, and is that a failure? When the optical path is fully blocked, the receiver never even detects the synchronization pulse,

so it can't recover any payload or compute a feature vector — there's nothing to classify. This isn't a classification error; it's a structural limitation of a system whose feature extraction depends on successful synchronization. It highlights a good direction for future work (a separate lightweight "is there any signal at all" detector).

Q55. If you deployed this in the real world, what would change? You'd want more diverse training data (varying distances, angles, actual outdoor ambient conditions), possibly a 5th "no signal / total blackout" class, and eventually corrective actions (like auto-alignment motors or automatic gain/power control) instead of only diagnosis.

Quick One-Liners (for rapid-fire questions)

Question	One-line Answer
Modulation used?	On-Off Keying (OOK)
Line coding?	NRZ-L
Bit rate?	1000 bits/sec (1ms/bit)
Samples per bit?	5, middle 60% of bit window, majority vote
Detection threshold?	1700 ADC counts
Sync pulse duration?	100ms, followed by 20ms guard interval
Transmitter MCU?	ESP32
Receiver MCU?	ESP32-S3
Laser?	650nm HW-493 module, driven via 2N2222 transistor
Photodiode?	BPW34 PIN photodiode, reverse-biased
Amplifier?	LM358, 100k Ω feedback resistor, 10nF (103) caps for filtering/decoupling
Features used?	Signal Voltage, BER, SNR, Ambient Light
Classes?	Normal, Optical Filter Attenuation, High Ambient Light, Severe Obstruction/Misalignment
Model type?	Compact neural network, TensorFlow Lite Micro
FP32 accuracy?	78.48%
INT8 quantized accuracy?	74.68%

Question	One-line Answer
Model size?	3.32 KB
Baseline comparison?	Random Forest, 77.22%
Biggest weakness?	Cannot classify complete signal loss (no sync = no features)